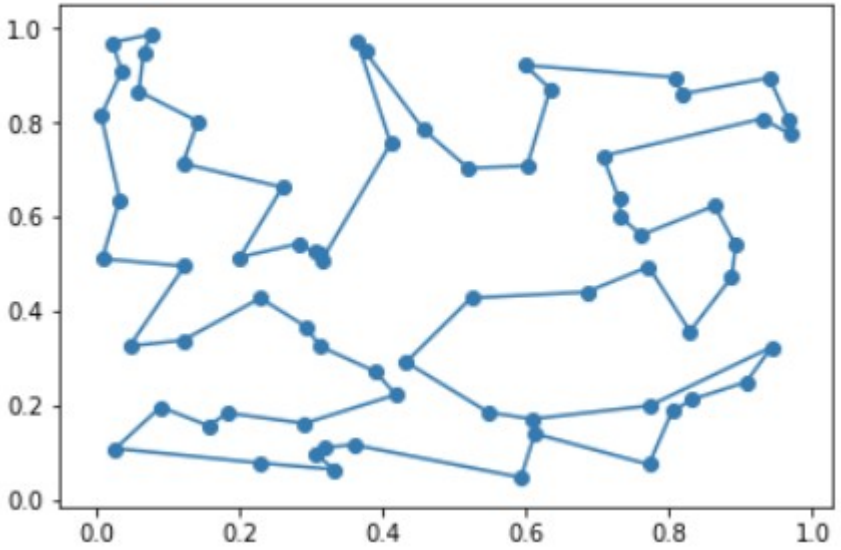
Solución del Problema del Viajero con Enfriamiento Simulado

Inteligencia Artificial

2021-II

Enunciado:

El problema del viajero (Travelling Salesman Problem, TSP) es un problema clásico entre los problemas de optimización, consiste en: dadas N ciudades, un agente viajero debe encontrar la ruta más corta posible que visite cada ciudad una sola vez en un mapa, regresando a la ciudad de origen.



Solución

Tendremos en cuenta los tres elementos para la representación y solución del problema.

- **Representación de una posible solución del problema**
- **Evaluación de cada una de las posibles soluciones**
- **Generación de nuevas posibles soluciones**

Representación de una posible solución del problema

No importa el método que utilice una solución del problema debe ser un listado de las ciudades del problema en el orden en el que se deben recorrer para obtener la mínima distancia total del problema:

Ejemplo:

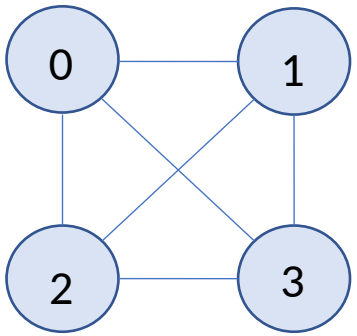
1) London 3) Dunedin 5) Beijing 7) Tokyo
2) Venice 4) Singapore 6) Phoenix 8) Victoria

Ruta 1 (3 5 7 2 1 6 4 8)
Ruta 2 (2 5 7 6 8 1 3 4)

Dos posible rutas

Evaluación de cada una de las posibles soluciones

El objetivo final del problema es encontrar el orden en que se debe visitar las ciudades con una mínima distancia total. Si tenemos las coordenadas de las ciudades podemos usar:



Ciudad	Coord x	Coord y
0	0.5	0.2
1	1	0.35
2	0.25	1
3	0.3	0.15

Ruta: (2, 1, 0, 3) Vector R

$p_1=(x_1, y_1)$ Ciudad 1 $p_2=(x_2, y_2)$ Ciudad 2

Función objetivo

Distancia Euclidiana:

$$D(p_1, p_2)=\sqrt{(x_1-x_2)^2+(y_1-y_2)^2}$$

Generación de nuevas posibles soluciones

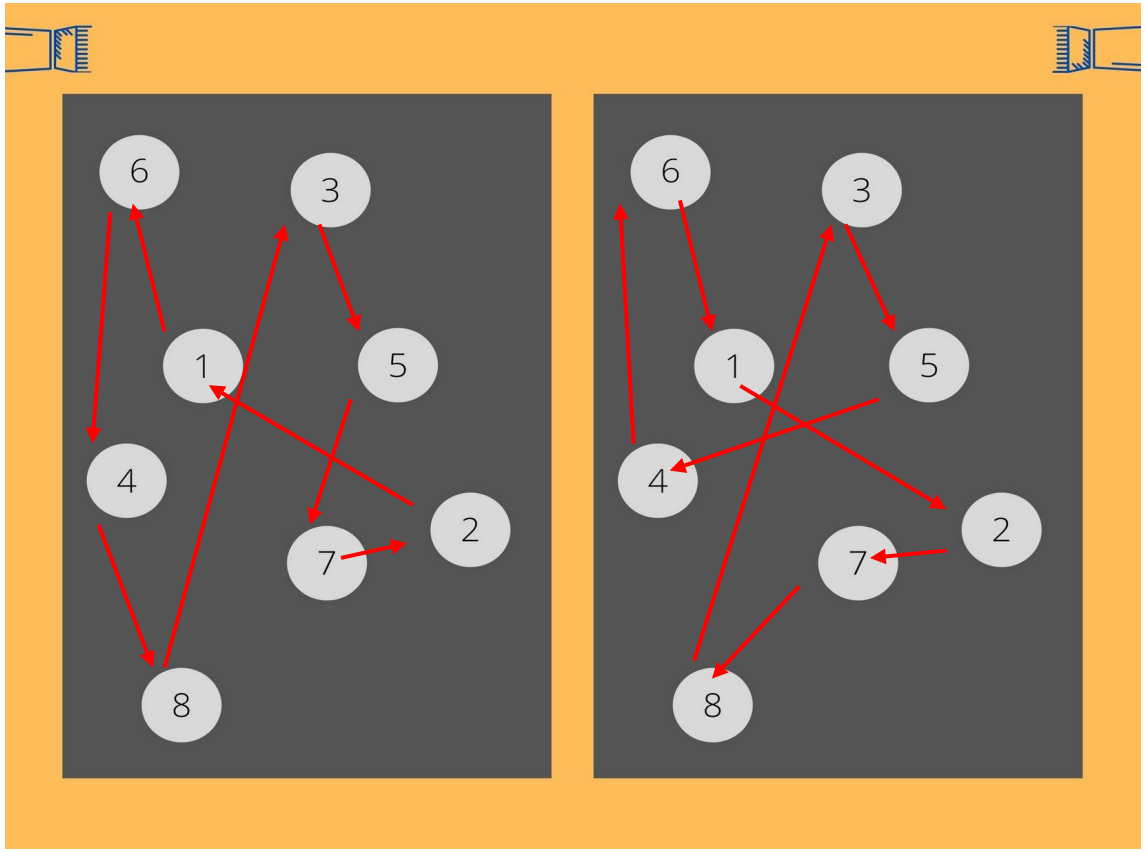
Dada una posible solución, una ruta se puede generar otra ruta con solo hacer permutaciones en el orden de las ciudades:

Ejemplo 2:

Ruta actual (3 5 7 2 1 6 4 8)

$i=2$ Dos índices al azar

Ruta generada (3 5 4 6 1 2 7 8)



Función principal de la búsqueda

Simulated_Anealing_Algorithm
currentNode = startNode;
= *iniTemp*
While >= *minTemp*
L = NEIGHBORS(currentNode);
nextNode = random(L);
If EVAL(nextNode) <= EVAL(currentNode)
currentNode = nextNode;
Else
If exp(- (nextEval - currentEval)/) >= random(0, 1)
currentNode = nextNode

```python
def anneal(self):
    """
    Execute simulated annealing algorithm.
    """
    # Initialize with the greedy solution.
    self.cur_solution, self.cur_fitness = self.initial_solution()

    print("Starting annealing.")
    while self.T >= self.stopping_temperature and
          self.iteration < self.stopping_iter:

        candidate = list(self.cur_solution)
        l = random.randint(2, self.N - 1)
        i = random.randint(0, self.N - l)
        candidate[i : (i + l)] = reversed(candidate[i : (i + l)])
        self.accept(candidate)
        self.T *= self.alpha
        self.iteration += 1

        self.fitness_list.append(self.cur_fitness)

    print("Best fitness obtained: ", self.best_fitness)
    improvement = 100 * (self.fitness_list[0] - self.best_fitness) /
                  (self.fitness_list[0])
    print(f"Improvement over greedy heuristic: {improvement : .2f}%")
```

```python
def p_accept(self, candidate_fitness):
    """
    Probability of accepting if the candidate is worse
    Depends on the current temperature and difference
    """
    return math.exp(-abs(candidate_fitness - self.cur_fitness) / self.T)

def accept(self, candidate):
    """
    Accept with probability 1 if candidate is better than current.
    Accept with probabilty p_accept(..) if candidate is worse.
    """
    candidate_fitness = self.fitness(candidate)
    if candidate_fitness < self.cur_fitness:
        self.cur_fitness, self.cur_solution = candidate_fitness, candidate
        if candidate_fitness < self.best_fitness:
            self.best_fitness, self.best_solution = candidate_fitness, candidate
    else:
        if random.random() < self.p_accept(candidate_fitness):
            self.cur_fitness, self.cur_solution = candidate_fitness, candidate
```

$$\exp\left(-\frac{|f(s)-f(t)|}{Temp}\right)$$

Boltzmann distribution

```python
def dist(self, node_0, node_1):
    """
    Euclidean distance between two nodes.
    """
    coord_0, coord_1 = self.coords[node_0], self.coords[node_1]
    return math.sqrt((coord_0[0] - coord_1[0]) ** 2 +
                     (coord_0[1] - coord_1[1]) ** 2)

def fitness(self, solution):
    """
    Total distance of the current solution path.
    """
    cur_fit = 0
    for i in range(self.N):
        cur_fit += self.dist(solution[i % self.N], solution[(i + 1) % self.N])
    return cur_fit
```

Distancia Euclidiana:

$$D(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$D_{total} = \left(\sum_{i=0}^{n-1} D(p_i, p_{i+1})\right) + D(p_n, p_0)$$

Función objetivo

Tarea

1. Hacer seguimiento a la implementación de SA en:

https://github.com/chncyhn/simulated-annealing-tsp

Parámetros

```
class SimAnneal(object):
    def __init__(self, coords, T=-1, alpha=-1, stopping_T=-1, stopping_iter=-1):
        self.coords = coords
        self.N = len(coords)
        self.T = math.sqrt(self.N) if T == -1 else T
        self.T_save = self.T  # save inital T to reset if batch annealing is used
        self.alpha = 0.995 if alpha == -1 else alpha
        self.stopping_temperature = 1e-8 if stopping_T == -1 else stopping_T
        self.stopping_iter = 100000 if stopping_iter == -1 else stopping_iter
        self.iteration = 1

        self.nodes = [i for i in range(self.N)]

        self.best_solution = None
        self.best_fitness = float("Inf")
        self.fitness_list = []
```

Temperatura inicial = 6.7
Tasa de enfriamiento = 0.995
Temperatura mínima = 1e-8